

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«МОСКОВСКИЙ ПОЛИТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ»
(МОСКОВСКИЙ ПОЛИТЕХ)

КУРСОВОЙ ПРОЕКТ

по курсу
«Разработка веб-приложений»

ТЕМА

«Разработка веб-приложения для подсчета калорий и анализа
питания пользователей»

Выполнил: Комар Ярослава-Прасковья Руслановна
Группа: 241-3211

Проверил: Кружалов Алексей Сергеевич

Москва, 2026

**«МОСКОВСКИЙ ПОЛИТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ»
(МОСКОВСКИЙ ПОЛИТЕХ)**

УТВЕРЖДАЮ
заведующая кафедрой
«Инфокогнитивные технологии»

_____ / Е. А. Пухова /

«____» _____ 2026 г.

ЗАДАНИЕ

на выполнение курсовой работы (проекта)

Комар Ярославе-Прасковье Руслановне,

обучающейся группы 241-3211,

направления подготовки 09.03.01 «Информатика и вычислительная техника»

по дисциплине «Разработка веб-приложений»

на тему «Разработка веб-приложения для подсчета калорий и анализа питания пользователей»

1. Исходные данные к работе (проекту): информационные ресурсы в сети интернет, научные публикации в открытой печати.

2. Содержание задания по курсовой работе (проекту) – перечень вопросов, подлежащих разработке:

Разрабатываемый вопрос	Объем, %	Срок	Примечание
Раздел 1. Анализ предметной области			
Задача 1.1. Обзор существующих программных продуктов по теме работы	10	05.04.2026	
Задача 1.2. Анализ программных инструментов разработки веб-приложений	7	05.04.2026	
Задача 1.3. Формулировка цели и задач работы	8	05.04.2026	
Раздел 2. Проектирование веб-приложения			
Задача 2.1. Анализ целевой аудитории	5	05.04.2026	
Задача 2.2. Описание функциональности приложения (диаграмма вариантов использования, user story)	7	10.04.2026	

Разрабатываемый вопрос	Объем, %	Срок	Примечание
Задача 2.3. Проектирование модели данных (ER-диаграмма, логическая и физическая схемы БД)	8	10.04.2026	
Задача 2.4. Разработка макетов страниц (Wireframe)	5	14.04.2026	
Раздел 3. Разработка веб-приложения			
Задача 3.1. Разработка структуры Flask-приложения	8	10.04.2026	
Задача 3.2. Реализация аутентификации и авторизации	7	05.05.2026	
Задача 3.3. Реализация системы ролей (user/admin)	7	05.05.2026	
Задача 3.4. CRUD-интерфейс (продукты, приёмы пищи)	12	30.05.2026	
Задача 3.5. Агрегация данных (калории, БЖУ)	6	30.05.2026	
Раздел 4. Оформление итогов работы			
Задача 4.1. Создание Git-репозитория с кодом проекта	3	03.04.2026	
Задача 4.2. Деплой приложения на хостинг	4	16.06.2026	
Задача 4.3. Оформление отчёта о проделанной работе	3	16.06.2026	

Руководитель курсовой работы (проекта):
старший преподаватель кафедры «Инфокогнитивные технологии»

«___» _____ 2026 г. _____ А. С. Кружалов
(подпись)

Дата выдачи задания «02» апреля 2026 г.

Дата сдачи выполненной работы «___» _____ 2026 г.

Задание принял к исполнению

«02» апреля 2026 г. _____ Комар Я-П.Р
(подпись)

СОДЕРЖАНИЕ

ВВЕДЕНИЕ	5
ГЛАВА 1. АНАЛИЗ ПРЕДМЕТНОЙ ОБЛАСТИ	6
1.1 Обзор существующих приложений для учета питания	6
1.2 Анализ программных инструментов разработки	7
1.3 Формулировка цели и задач работы	8
ГЛАВА 2. ПРОЕКТИРОВАНИЕ ВЕБ-ПРИЛОЖЕНИЯ	10
2.1 Анализ целевой аудитории	10
2.2 Описание функциональности приложения	10
2.3 Проектирование модели данных	11
2.4 Разработка макетов страниц	16
ГЛАВА 3. РАЗРАБОТКА ВЕБ-ПРИЛОЖЕНИЯ	19
3.1 Реализация моделей базы данных в Django	19
3.2 Реализация расчёта норм в модели User	20
3.3 Реализация API-эндпоинтов	21
3.4 Фронтенд-логика на JavaScript	23
3.5 Настройка административной панели	25
3.6 Итоги разработки	26
ЗАКЛЮЧЕНИЕ	28

ВВЕДЕНИЕ

Актуальность темы. В условиях роста интереса к здоровому образу жизни и правильному питанию возрастает потребность в удобных инструментах для контроля рациона. Традиционные методы ведения дневников питания (бумажные блокноты или таблицы) неудобны, требуют значительных временных затрат и не дают оперативной аналитики. Разработка специализированного веб-приложения позволяет автоматически подсчитывать калорийность блюд, анализировать баланс белков, жиров и углеводов (БЖУ), а также отслеживать динамику питания.

Степень разработанности проблемы. Вопросы анализа питания и расчета энергетической ценности продуктов активно исследуются в области нутрициологии и информационных технологий. В области веб-ориентированных систем существенный вклад внесли создатели популярных приложений (например, FatSecret, Lifesum). Однако многие из них имеют ограничения, связанные с платным доступом, недостаточной адаптацией под индивидуальные потребности пользователей или ограниченной гибкостью. Разработка доступных, расширяемых и адаптивных веб-приложений остается актуальной задачей.

Объект исследования — процессы учета и анализа питания пользователей в рамках цифровых систем.

Предмет исследования — программные средства и алгоритмы, обеспечивающие автоматизацию подсчета калорий, анализ макронутриентов и визуализацию данных в веб-приложении.

Целью данной работы является проектирование и разработка веб-приложения, предназначенного для учета рациона питания пользователей, автоматического расчета калорийности и анализа структуры питания с целью формирования более сбалансированных пищевых привычек.

ГЛАВА 1. АНАЛИЗ ПРЕДМЕТНОЙ ОБЛАСТИ

1.1 Обзор существующих приложений для учета питания

На современном рынке цифровых решений для контроля питания можно выделить две основные группы: мобильные приложения с облачной синхронизацией и веб-ориентированные платформы для анализа рациона. Проведем сравнение наиболее популярных решений в таблице 1.1.

Таблица 1.1 – Сравнительный анализ приложений для учета питания

Критерий	MyFitnessPal	Yazio	FatSecret
Ключевая особенность	Обширная база продуктов	Планы питания	Соц. функции
Модель доступа	Freemium	Freemium	Бесплатно
Наличие рекламы	Есть	Есть	Нет
Готовые планы питания	Только премиум	Премиум	Нет
Графики и аналитика	Базовая	Детальная	Базовая
Русскоязычный интерфейс	Нет	Да	Да
Социальные функции	Да	Нет	Да

Общие недостатки существующих решений:

- Большинство приложений используют модель Freemium — расширенные функции (детальная аналитика, планы питания) доступны только за плату;
- Наличие рекламы в бесплатных версиях, что ухудшает пользовательский опыт;
- Отсутствие гибкости настройки под индивидуальные потребности пользователя;
- Ограниченные возможности по управлению и расширению справочника продуктов через веб-интерфейс.

Актуальность разработки собственного решения:

Таким образом, создание веб-приложения для подсчета калорий и анализа питания, которое будет полностью бесплатным, не содержать рекламы, предоставлять возможность управления справочником продуктов через административную панель и обеспечивать базовую аналитику по калорийности и БЖУ, является актуальной задачей. Разрабатываемое приложение также

позволит пользователю вести дневник питания без ограничений и адаптировать систему под свои индивидуальные потребности.

1.2 Анализ программных инструментов разработки

Для реализации проекта рассматриваются современные подходы к разработке веб-приложений с клиент-серверной архитектурой. Выбор инструментов обусловлен требованиями к удобству интерфейса, производительности и масштабируемости системы:

- **Frontend-фреймворк:** Использование тяжелых фронтенд-фреймворков (React, Vue.js) и подход на основе чистого JavaScript с Bootstrap. React и Vue.js являются мощными инструментами, предоставляющими компонентную архитектуру, виртуальный DOM и богатые экосистемы. Однако их применение в данном проекте признано избыточным по следующим причинам: интерфейс дневника питания представляет собой совокупность статичных страниц с ограниченной динамикой; использование React или Vue.js потребовало бы настройки системы сборки (Webpack, Vite), изучения дополнительных концепций (хуки, маршрутизация) и увеличило бы время разработки без существенного выигрыша в функциональности. В связи с этим выбран подход на основе HTML, CSS, Bootstrap и чистого JavaScript.
- **Backend-платформа:** Для реализации серверной части был проведен сравнительный анализ FastAPI, Flask и Django. FastAPI является современным асинхронным фреймворком с высокой производительностью и автоматической генерацией документации, однако он не предоставляет встроенной ORM, административной панели и системы аутентификации, что потребовало бы написания значительного объема дополнительного кода. Flask — легковесный микрофреймворк, обладающий гибкостью и большим количеством расширений, но также требующий самостоятельной реализации многих компонентов. Django, в отличие от них, предлагает встроенную ORM для удобной работы с PostgreSQL, готовую административную панель для управления справочником продуктов, встроенную систему аутентификации пользователей и механизм миграций для автоматического обновления структуры базы данных. Эти

преимущества существенно ускоряют разработку и делают Django оптимальным выбором для данного проекта. Использование встроенной ORM позволяет эффективно обрабатывать запросы от множества пользователей при одновременном подсчете калорий и агрегации данных о питании за длительные периоды. Bootstrap обеспечивает быструю адаптивную верстку и готовые компоненты интерфейса (формы, таблицы, карточки), а чистый JavaScript позволяет реализовать необходимую динамику: отправку асинхронных запросов к серверу, динамическое обновление таблицы приемов пищи и валидацию вводимых данных без избыточного усложнения проекта.

- **Хранение данных:** Для хранения данных рассматривались реляционная СУБД PostgreSQL и легковесная SQLite. SQLite не требует установки и настройки, что удобно для прототипирования, однако имеет ограниченную поддержку сложных запросов, снижает производительность при росте объема данных и не рассчитана на высокие нагрузки. PostgreSQL, напротив, является промышленным стандартом, обеспечивает полную поддержку транзакций, сложных агрегирующих запросов (необходимых для расчета суточной калорийности и статистики питания), надежное хранение данных и масштабируемость.

Для реализации клиентской части приложения выбран подход на основе HTML, CSS, Bootstrap и чистого JavaScript. При создании интерфейсов использовались официальные руководства из документации разработчиков [1].

Для реализации серверной части приложения выбран фреймворк Django (Python), архитектура которого подробно описана в работе [2]. Использование встроенной ORM позволяет эффективно обрабатывать запросы от множества пользователей при одновременном подсчете калорий и агрегации данных о питании.

1.3 Формулировка цели и задач работы

На основе проведенного анализа инструментов (см. подраздел 1.2) и выявленных пробелов в существующих продуктах, была сформулирована цель исследования.

Цель работы — создание веб-приложения для учета потребляемых

продуктов, автоматического подсчета калорий и анализа структуры питания пользователей.

Для достижения поставленной цели необходимо решить следующие задачи:

1. Провести сравнительный анализ существующих приложений для учета питания и методов расчета суточной нормы калорий;
2. Разработать архитектуру клиент-серверного приложения и спроектировать структуру базы данных PostgreSQL для хранения информации о пользователях, продуктах и истории приемов пищи;
3. Реализовать программные модули для ведения дневника питания (добавление/удаление записей) и мониторинга суточной калорийности;
4. Интегрировать справочник продуктов и внедрить алгоритм автоматического подсчета КБЖУ на основе веса порции;
5. Провести тестирование разработанной системы на корректность расчета калорий и удобство работы с дневником питания.

ГЛАВА 2. ПРОЕКТИРОВАНИЕ ВЕБ-ПРИЛОЖЕНИЯ

2.1 Анализ целевой аудитории

Основными пользователями разрабатываемого веб-приложения являются люди, стремящиеся контролировать свое питание и вести здоровый образ жизни. Целевую аудиторию можно разделить на три основных сегмента:

- **Худеющие люди** — хотят снизить вес, контролируют калории, для них важен точный подсчет КБЖУ и дефицит калорий.
- **Спортсмены** — контролируют баланс белков, жиров и углеводов для набора массы или "сушки".

Выделяются две ключевые роли в системе:

- **Обычный пользователь** — человек, ведущий дневник питания, добавляющий продукты в свой рацион и отслеживающий КБЖУ. Основные задачи: удобное добавление приемов пищи, просмотр статистики за день/неделю, контроль выполнения суточной нормы калорий.
- **Администратор системы** — пользователь с расширенными правами, ответственный за поддержание справочника продуктов в актуальном состоянии. Основные задачи: добавление новых продуктов, редактирование существующих, удаление устаревших позиций через встроенную административную панель Django.

Для каждой роли проведено профилирование по методу персонажей (Personas) для минимизации когнитивной нагрузки на интерфейс.

2.2 Описание функциональности приложения

Функциональные возможности системы определяются через взаимодействие пользователя с системой. Основной бизнес-кейс представлен в виде User Story: «Я, как пользователь, хочу добавить съеденный продукт с указанием его веса, чтобы система автоматически рассчитала количество калорий и БЖУ для данного приема пищи».

На рисунке 2.1 представлена диаграмма вариантов использования (Use Case).



Рисунок 2.1 – Диаграмма вариантов использования системы

2.3 Проектирование модели данных

При выборе архитектуры базы данных был проведен сравнительный анализ различных СУБД. В результате принято решение использовать PostgreSQL как промышленный стандарт, обеспечивающий надежность хранения данных, поддержку транзакций и возможность выполнения сложных агрегирующих запросов для расчета суточной калорийности и статистики питания.

Структура базы данных разрабатываемого приложения включает сле-

дующие основные таблицы: users (пользователи), products (справочник продуктов), meal_entries (записи о приемах пищи).

Структура таблицы users (пользователи системы) представлена в табл. 2.1.

Таблица 2.1 – Структура таблицы пользователей (users)

Поле	Тип	Ограничение	Описание
id	SERIAL	PRIMARY KEY	Уникальный идентификатор пользователя
username	VARCHAR(50)	NOT NULL, UNIQUE	Логин пользователя
password	VARCHAR(255)	NOT NULL	Хеш пароля
email	VARCHAR(100)	UNIQUE	Адрес электронной почты
birth_date	DATE	NOT NULL	Дата рождения (для расчета возраста)
gender	VARCHAR(10)	NOT NULL	Пол (male/female)
activity_level	VARCHAR(20)	NOT NULL	Уровень активности
goal	VARCHAR(20)	DEFAULT 'maintenance'	Цель (lose/maintain/gain)
weight	NUMERIC(5,1)	NULL	Вес пользователя (кг)
height	INT	NULL	Рост пользователя (см)
daily_calorie_norm	INT	NULL	Суточная норма калорий
daily_protein_norm	INT	NULL	Дневная норма белка, г
daily_fat_norm	INT	NULL	Дневная норма жиров, г
daily_carbohydrates_norm	INT	NULL	Дневная норма углеводов, г
is_admin	BOOLEAN	DEFAULT FALSE	Флаг администратора
created_at	TIMESTAMP	DEFAULT NOW()	Дата регистрации
updated_at	TIMESTAMP	DEFAULT NOW()	Дата последнего обновления

Значения поля goal:

- **lose** — похудение (дефицит калорий, -15% от нормы);
- **maintain** — поддержание веса (норма калорий);
- **gain** — набор массы (профицит калорий, +15% от нормы).

Структура таблицы `products` представлена в табл. 2.2.

Таблица 2.2 – Структура таблицы продуктов (`products`)

Поле	Тип	Ограничение	Описание
<code>id</code>	<code>SERIAL</code>	<code>PRIMARY KEY</code>	Уникальный идентификатор продукта
<code>name</code>	<code>VARCHAR(100)</code>	<code>NOT NULL</code>	Название продукта
<code>calories</code>	<code>NUMERIC(6,2)</code>	<code>NOT NULL</code>	Калорийность на 100 г, ккал
<code>protein</code>	<code>NUMERIC(5,2)</code>	<code>NOT NULL</code>	Белки на 100 г, г
<code>fat</code>	<code>NUMERIC(5,2)</code>	<code>NOT NULL</code>	Жиры на 100 г, г
<code>carbohydrates</code>	<code>NUMERIC(5,2)</code>	<code>NOT NULL</code>	Углеводы на 100 г, г
<code>created_at</code>	<code>TIMESTAMP</code>	<code>DEFAULT NOW()</code>	Дата добавления продукта
<code>updated_at</code>	<code>TIMESTAMP</code>	<code>DEFAULT NOW()</code>	Дата последнего изменения

Структура таблицы `meal_entries` (записи о приемах пищи) представлена в табл. 2.3.

Таблица 2.3 – Структура таблицы приемов пищи (`meal_entries`)

Поле	Тип	Ограничение	Описание
<code>id</code>	<code>SERIAL</code>	<code>PRIMARY KEY</code>	Уникальный идентификатор записи
<code>user_id</code>	<code>INT</code>	<code>FOREIGN KEY (users.id)</code>	Ссылка на пользователя

Продолжение таблицы 2.3

Поле	Тип	Ограничение	Описание
product_id	INT	FOREIGN KEY (products.id)	Ссылка на продукт
weight	NUMERIC(6,1)	NOT NULL	Вес порции, грамм
meal_type	VARCHAR(20)	NOT NULL	Тип приема пищи (завтрак/обед/ужин/перекус)
consumed_at	TIMESTAMP	NOT NULL	Дата и время приема пищи
created_at	TIMESTAMP	DEFAULT NOW()	Время создания записи в системе

ER-диаграмма базы данных

На основе спроектированных таблиц была разработана ER-диаграмма, отображающая сущности базы данных и связи между ними. Связи между сущностями:

- users и meal_entries — связь один ко многим (один пользователь может иметь много записей о приемах пищи);
- products и meal_entries — связь один ко многим (один продукт может использоваться во многих записях).

Калорийность порции рассчитывается следующим образом:

$$K = \frac{K_{100} \cdot W}{100} \quad (1)$$

где

K — калорийность порции, ккал;

K_{100} — калорийность продукта на 100 г, ккал;

W — вес порции, грамм.

Аналогично рассчитываются белки, жиры и углеводы для порции:

$$P = \frac{P_{100} \cdot W}{100} \quad (2)$$

где

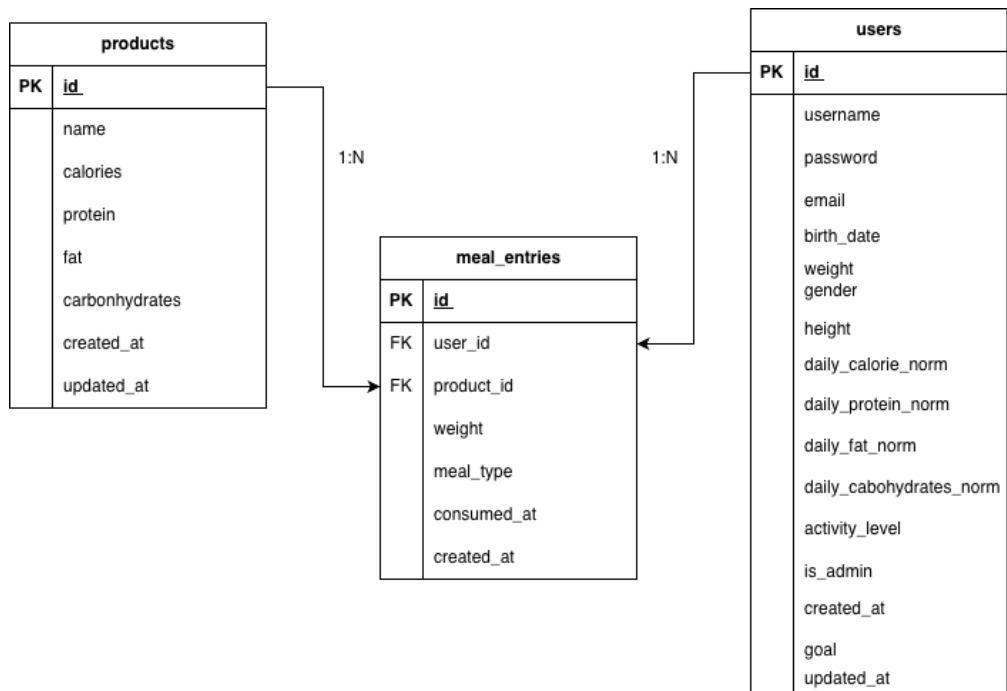


Рисунок 2.2 – ER-диаграмма БД

P — содержание белков в порции, г;

P_{100} — содержание белков в продукте на 100 г, г;

W — вес порции, грамм.

Суточная норма калорий рассчитывается по формуле Миффлина-Сан-Жеора:

Для женщин:

$$BMR = 10 \cdot W + 6.25 \cdot H - 5 \cdot A - 161 \quad (3)$$

Для мужчин:

$$BMR = 10 \cdot W + 6.25 \cdot H - 5 \cdot A + 5 \quad (4)$$

где

BMR — базальный уровень метаболизма (основной обмен), ккал/сут;

W — вес пользователя, кг;

H — рост пользователя, см;

A — возраст пользователя, лет.

Итоговая суточная норма калорий с учетом физической активности и

цели:

$$Daily_calories = BMR \cdot Activity_factor \cdot Goal_factor \quad (5)$$

где

Activity_factor — коэффициент активности (от 1.2 до 2.0);

Goal_factor — коэффициент цели:

- для похудения (lose): 0.85 (дефицит 15%);
- для поддержания (maintain): 1.0;
- для набора массы (gain): 1.15 (профицит 15%).

2.4 Разработка макетов страниц

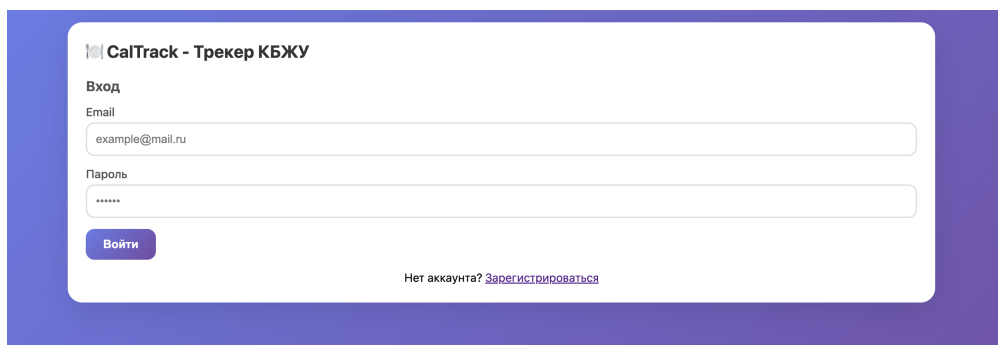


Рисунок 2.3 – Макет страницы входа

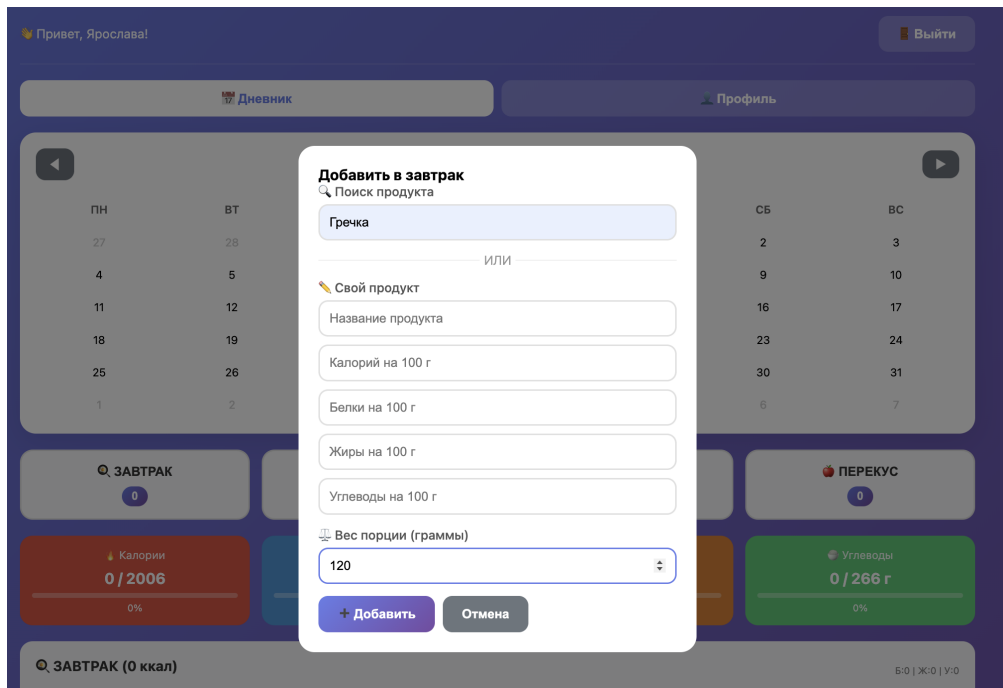


Рисунок 2.6 – Макет добавления блюда

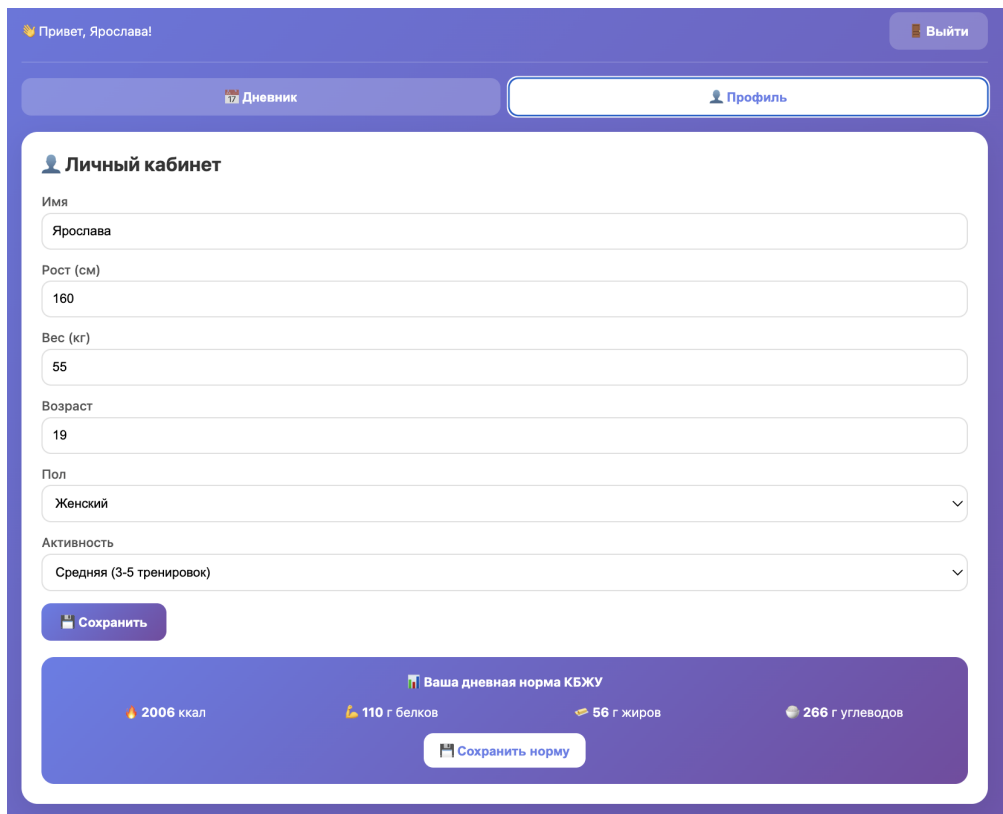


Рисунок 2.7 – Макет страницы пользователя

ГЛАВА 3. РАЗРАБОТКА ВЕБ-ПРИЛОЖЕНИЯ

3.1 Реализация моделей базы данных в Django

На основе спроектированной в главе 2 структуры базы данных были созданы модели Django, соответствующие таблицам User, Product и MealEntry. Модель User расширяет стандартного пользователя Django через AbstractUser, добавляя поля для хранения антропометрических данных, уровня активности, цели и рассчитанных норм КБЖУ. Модели Product и MealEntry реализуют справочник продуктов и записи о приёмах пищи; вычисляемые поля БЖУ реализованы через свойства (@property). Листинг 1 демонстрирует фрагменты моделей Product и MealEntry.

Листинг 1 – Модели Product и MealEntry

```
1 class Product(models.Model):
2     name = models.CharField(max_length=100, unique=True)
3     calories = models.DecimalField(max_digits=6,
4                                     decimal_places=2)
5     protein = models.DecimalField(max_digits=5,
6                                    decimal_places=2)
7     fat = models.DecimalField(max_digits=5, decimal_places=
8                                =2)
9     carbohydrates = models.DecimalField(max_digits=5,
10                                         decimal_places=2)
11
12 class MealEntry(models.Model):
13     MEAL_TYPE_CHOICES = [
14         ('breakfast', 'Завтрак'), ('lunch', 'Обед'),
15         ('dinner', 'Ужин'), ('snack', 'Перекус'),
16     ]
17     user = models.ForeignKey(User, on_delete=models.CASCADE)
18     product = models.ForeignKey(Product, on_delete=models.
19                                CASCADE)
20     weight = models.DecimalField(max_digits=6,
21                                   decimal_places=1)
22     meal_type = models.CharField(max_length=10, choices=
23                                   MEAL_TYPE_CHOICES)
24     consumed_at = models.DateTimeField()
25
26     @property
27     def total_calories(self):
28         return round(float(self.product.calories) * float(
29             self.weight) / 100, 1)
```

```

22
23     @property
24     def total_protein(self):
25         return round(float(self.product.protein) * float(
            self.weight) / 100, 1)

```

3.2 Реализация расчёта норм в модели User

В модели User реализованы методы расчёта суточной нормы калорий и норм белков, жиров, углеводов. Листинг 2 демонстрирует реализацию формулы Миффлина–Сан-Жеора с учётом коэффициентов активности и цели. При сохранении профиля нормы автоматически пересчитываются в методе save().

Листинг 2 – Методы расчёта норм КБЖУ

```

1 def calculate_daily_calorie_norm(self):
2     """Расчёт суточной нормы по формуле МиффлинаСанЖеора--."""
3     if not all([self.weight, self.height, self.birth_date,
4                 self.gender]):
5         return None
6     age = self.get_age()
7     w = float(self.weight)
8     h = float(self.height)
9     if self.gender == 'male':
10         bmr = 10 * w + 6.25 * h - 5 * age + 5
11     else:
12         bmr = 10 * w + 6.25 * h - 5 * age - 161
13     activity = float(self.activity_level)
14     tdee = bmr * activity
15     goal_mult = {'lose': 0.85, 'maintain': 1.0, 'gain':
16                  1.15}.get(self.goal, 1.0)
17     return round(tdee * goal_mult)
18
19 def calculate_daily_norms(self):
20     calories = self.calculate_daily_calorie_norm()
21     if calories is None or not self.weight:
22         return None, None, None, None
23     w = float(self.weight)
24     protein_per_kg = {'lose': 1.8, 'gain': 1.8, 'maintain':
25                      1.6}.get(self.goal, 1.6)
26     protein = round(w * protein_per_kg)
27     fat = round(calories * 0.25 / 9)
28     carbs = round((calories - protein * 4 - fat * 9) / 4)

```

```

26     carbs = max(carbs, 0)
27     return calories, protein, fat, carbs
28
29 def save(self, *args, **kwargs):
30     cal, prot, fat, carbs = self.calculate_daily_norms()
31     if cal is not None:
32         self.daily_calorie_norm = cal
33         self.daily_protein_norm = prot
34         self.daily_fat_norm = fat
35         self.daily_carbohydrates_norm = carbs
36     super().save(*args, **kwargs)

```

3.3 Реализация API-эндпоинтов

Для динамического обновления дневника питания созданы REST-подобные эндпоинты. Поиск продуктов выполняется по префиксу названия (`startswith`), что обеспечивает релевантное автодополнение. В листинге 3 представлен поиск продуктов, в листинге 4 — добавление записи, в листинге 5 — удаление записи.

Листинг 3 – API поиска продуктов

```

1 @login_required
2 @require_GET
3 def api_products_search(request):
4     q = request.GET.get('q', '').strip()
5     if len(q) < 1:
6         return JsonResponse([], safe=False)
7     products = Product.objects.filter(name__startswith=q).
order_by('name')[:100]
8     data = [
9         {
10             'id': p.id,
11             'name': p.name,
12             'calories': float(p.calories),
13             'protein': float(p.protein),
14             'fat': float(p.fat),
15             'carbohydrates': float(p.carbohydrates),
16         }
17         for p in products
18     ]
19     return JsonResponse(data, safe=False)

```

Листинг 4 – API добавления приёма пищи

```
1 @login_required
2 @require_POST
3 def api_meal_entry_add(request):
4     try:
5         data = json.loads(request.body)
6         product_id = data.get('product_id')
7         weight = data.get('weight')
8         meal_type = data.get('meal_type')
9         consumed_at = data.get('consumed_at')
10
11         if not all([product_id, weight, meal_type,
12 consumed_at]):
13             return JsonResponse({'error': 'Все поля
14 обязательны.'}, status=400)
15
16         product = get_object_or_404(Product, pk=product_id)
17         consumed_at_dt = datetime.fromisoformat(consumed_at)
18         if timezone.is_naive(consumed_at_dt):
19             consumed_at_dt = timezone.make_aware(
20 consumed_at_dt)
21
22         entry = MealEntry.objects.create(
23             user=request.user,
24             product=product,
25             weight=weight,
26             meal_type=meal_type,
27             consumed_at=consumed_at_dt,
28         )
29         return JsonResponse({
30             'id': entry.id,
31             'product_name': entry.product.name,
32             'weight': float(entry.weight),
33             'meal_type': entry.meal_type,
34             'total_calories': entry.total_calories,
35             'total_protein': entry.total_protein,
36             'total_fat': entry.total_fat,
37             'total_carbohydrates': entry.total_carbohydrates
38         }, status=201)
39     except (json.JSONDecodeError, ValueError) as e:
40         return JsonResponse({'error': str(e)}, status=400)
```

Листинг 5 – API удаления записи

```
1 @login_required
2 @require_http_methods(['DELETE'])
3 def api_meal_entry_delete(request, entry_id):
4     entry = get_object_or_404(MealEntry, pk=entry_id, user=
        request.user)
5     entry.delete()
6     return JsonResponse({'message': 'Запись удалена.'})
```

3.4 Фронтенд-логика на JavaScript

Клиентская динамика реализована на чистом JavaScript и разделена на три модуля: `api.js` — HTTP-запросы к серверу, `charts.js` — обновление итогов и прогресс-баров, `main.js` — логика страницы дневника. Листинг 6 показывает слой API, листинг 7 — поиск с автодополнением, листинг 8 — обновление итогов и удаление записи.

Листинг 6 – Модуль API-запросов (`api.js`)

```
1 async function apiFetch(url, options = {}) {
2     const headers = { ...(options.headers || {}) };
3     if (options.method && options.method !== 'GET') {
4         headers['X-CSRFToken'] = getCsrfToken();
5     }
6     return fetch(url, { ...options, headers });
7 }
8
9 async function searchProducts(q) {
10     const res = await apiFetch(`/api/products/search?q=${
        encodeURIComponent(q)}`);
11     return res.json();
12 }
13
14 async function addMealEntry(payload) {
15     const res = await apiFetch('/api/meal-entries/add', {
16         method: 'POST',
17         headers: { 'Content-Type': 'application/json' },
18         body: JSON.stringify(payload),
19     });
20     return { ok: res.ok, data: await res.json() };
21 }
22
23 async function deleteMealEntry(id) {
```

```

24   const res = await apiFetch(`/api/meal-entries/${id}/delete
    `, { method: 'DELETE' });
25   return res.ok;
26 }

```

Листинг 7 – Поиск продуктов с автодополнением (main.js)

```

1  let selectedProduct = null;
2  let searchTimer = null;
3
4  searchInput.addEventListener('input', function () {
5    clearTimeout(searchTimer);
6    const q = this.value.trim();
7    if (q.length < 1) { hideSuggestions(); return; }
8    searchTimer = setTimeout(async () => {
9      renderSuggestions(await searchProducts(q));
10   }, 250);
11 });
12
13 function selectProduct(p) {
14   selectedProduct = p;
15   searchInput.value = p.name;
16   document.getElementById('selectedProductId').value = p.id;
17   document.getElementById('selectedProductInfo').classList.
     remove('d-none');
18   updatePreview();
19 }
20
21 function updatePreview() {
22   const w = parseFloat(weightInput.value);
23   if (!selectedProduct || !w || w <= 0) return;
24   const r = w / 100;
25   document.getElementById('pvKcal').textContent =
26     Math.round(selectedProduct.calories * r) + ' ккал';
27   document.getElementById('pvProtein').textContent =
28     (selectedProduct.protein * r).toFixed(1) + 'г';
29   document.getElementById('nutrientPreview').classList.
     remove('d-none');
30 }

```

Листинг 8 – Обновление итогов и удаление записи

```

1  function updateTotals(cal, prot, fat, carbs) {
2    runCalories = Math.max(0, +(runCalories + cal).toFixed(1))

```



```

    ;
3   runProtein  = Math.max(0, +(runProtein  + prot).toFixed(1)
    );
4   runFat      = Math.max(0, +(runFat      + fat).toFixed(1))
    ;
5   runCarbs    = Math.max(0, +(runCarbs    + carbs).toFixed
    (1));
6
7   document.getElementById('totalCalories').textContent =
    runCalories;
8   setBar('caloriesBar', runCalories, NORM_CALORIES, true);
9   setBar('proteinBar',  runProtein,  NORM_PROTEIN);
10 }
11
12 function setBar(id, current, norm, colored) {
13   const bar = document.getElementById(id);
14   if (!bar || !norm) return;
15   const pct = Math.min(Math.round(current / norm * 100),
    100);
16   bar.style.width = pct + '%';
17   if (colored) {
18     bar.className = 'progress-bar ' +
19       (pct >= 100 ? 'bg-danger' : pct >= 80 ? 'bg-success' :
    'bg-primary');
20   }
21 }
22
23 async function deleteEntry(id, btn) {
24   const confirmed = await showConfirm(
25     'Запись' будет удалена из дневника без возможности
    восстановления.',
26     'Удалить' запись?');
27   );
28   if (!confirmed) return;
29   if (await deleteMealEntry(id)) {
30     document.getElementById('entry-row-${id}')?.remove();
31     updateTotals(-cal, -prot, -fat, -carbs);
32   }
33 }

```

3.5 Настройка административной панели

Административная панель Django настроена для управления пользователями, продуктами и записями о приёмах пищи (листинг 9).

Листинг 9 – Настройка админ-панели

```
1 @admin.register(User)
2 class UserAdmin(BaseUserAdmin):
3     list_display = ('username', 'email', 'gender', 'weight',
4                     'height',
5                     'goal', 'daily_calorie_norm', 'is_admin',
6                     'date_joined')
7     list_filter = ('is_admin', 'goal', 'activity_level', 'gender', 'is_active')
8     search_fields = ('username', 'email', 'first_name', 'last_name')
9     fieldsets = BaseUserAdmin.fieldsets + (
10         ('Профиль', {'fields': ('birth_date', 'gender', 'weight', 'height',
11                                 'goal', 'activity_level', 'daily_calorie_norm', 'is_admin')})),
12     )
13     readonly_fields = ('daily_calorie_norm', 'created_at', 'updated_at')
14
15 @admin.register(Product)
16 class ProductAdmin(admin.ModelAdmin):
17     list_display = ('name', 'calories', 'protein', 'fat', 'carbohydrates', 'created_at')
18     search_fields = ('name',)
19     ordering = ('name',)
20
21 @admin.register(MealEntry)
22 class MealEntryAdmin(admin.ModelAdmin):
23     list_display = ('user', 'product', 'weight', 'meal_type', 'consumed_at')
24     list_filter = ('meal_type', 'consumed_at')
25     search_fields = ('user__username', 'product__name')
26     date_hierarchy = 'consumed_at'
```

3.6 Итоги разработки

В результате выполнения работ по главе 3 было разработано веб-приложение для подсчёта калорий и анализа питания. Реализованы:

- модели базы данных User, Product, MealEntry с вычисляемыми полями БЖУ;
- автоматический расчёт суточной нормы калорий по формуле Миффлина–

Сан-Жеора;

- расчёт норм белков, жиров и углеводов с сохранением в профиле пользователя;
- API-эндпоинты для поиска продуктов, добавления и удаления записей;
- модульная клиентская логика (`api.js`, `charts.js`, `main.js`);
- динамический интерфейс с автодополнением, превью БЖУ и прогресс-барами;
- модальное окно подтверждения удаления в стиле приложения;
- административная панель для управления пользователями, продуктами и записями.

GitHub репозиторий с кодом проекта доступен по ссылке:

`github.com/derbara/Caltrack`

ЗАКЛЮЧЕНИЕ

В результате проведенного исследования подтверждена актуальность разработки веб-приложения для подсчета калорий и анализа питания. Практическая реализация показала, что автоматизация учета потребляемых продуктов и расчетов пищевой ценности позволяет пользователям эффективно контролировать свой рацион и достигать поставленных целей (похудение, поддержание веса, набор массы).

На основе полученных при разработке данных можно сформулировать следующие **выводы**:

1. Сравнительный анализ существующих приложений (MyFitnessPal, Yazio, FatSecret) выявил их основные недостатки: ограниченный бесплатный функционал, наличие рекламы и отсутствие гибкости настройки. Разработанное приложение полностью бесплатно, не содержит рекламы и предоставляет пользователю возможность управления справочником продуктов через административную панель.
2. Выбранный стек технологий (Django, PostgreSQL, Bootstrap, чистый JavaScript) показал свою эффективность для реализации данного класса задач. Встроенная ORM Django обеспечивает удобную работу с базой данных, а асинхронные JavaScript-запросы позволяют динамически обновлять дневник питания без перезагрузки страницы.
3. Реализованный алгоритм расчета суточной нормы калорий по формуле Миффлина-Сан-Жеора с учетом индивидуальных параметров пользователя (пол, вес, рост, возраст, уровень активности) и выбранной цели (дефицит 15%, поддержание, профицит 15%) обеспечивает персонализированные рекомендации. Расчет норм белков, жиров и углеводов дополняет аналитику и повышает ценность приложения для пользователя.

Рекомендации и предложения. Для практического использования разработанного приложения рекомендуется развертывание на облачной платформе. Это обеспечит круглосуточный доступ пользователей к своим дневникам питания с любого устройства. Также рекомендуется регулярное обновление

справочника продуктов через административную панель для поддержания актуальности базы данных.

Перспективы углубления темы. В дальнейшем исследовании планируется развитие функциональности системы за счет:

- внедрения сканера штрихкодов для быстрого добавления продуктов;
- разработки мобильного приложения (на базе React Native или Flutter);
- добавления функции импорта рецептов и автоматического расчета КБЖУ блюд;
- интеграции с фитнес-трекерами (Google Fit, Apple Health) для учета сожженных калорий;
- использования методов машинного обучения для персонализированных рекомендаций по питанию.

При разработке пояснительной записки к курсовому проекту строго соблюдались требования [?], определяющие структуру и правила оформления научно-исследовательских работ.

СПИСОК ЛИТЕРАТУРЫ

- [1] Bootstrap 5.3 Documentation: Introduction [Электронный ресурс]. — [Б. м. : б. и.]. — URL: <https://getbootstrap.com/docs/5.3/getting-started/introduction/> (дата обращения: 24.04.2026).
- [2] Django 6.0 Documentation [Электронный ресурс]. — [Б. м. : б. и.]. — URL: <https://docs.djangoproject.com/en/6.0/> (дата обращения: 24.04.2026).